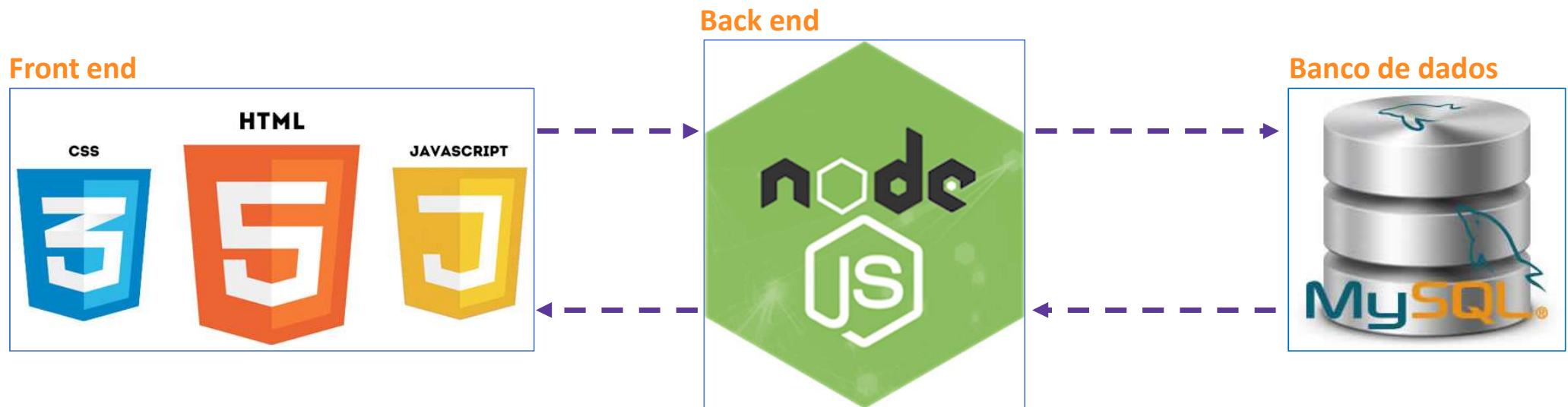


NODE JS



NODE JS



Node.js é um interpretador de [JavaScript](#) assíncrono com código aberto orientado a eventos,, focado em migrar a programação do Javascript do cliente (*frontend*) para os servidores, criando aplicações de alta escalabilidade (como um servidor web), manipulando milhares de conexões/eventos simultâneas em tempo real.

NODE JS



O [Node.js](https://nodejs.org/) é um ambiente Javascript desenvolvido para ser executado em servidores e possui algumas características bem interessantes:

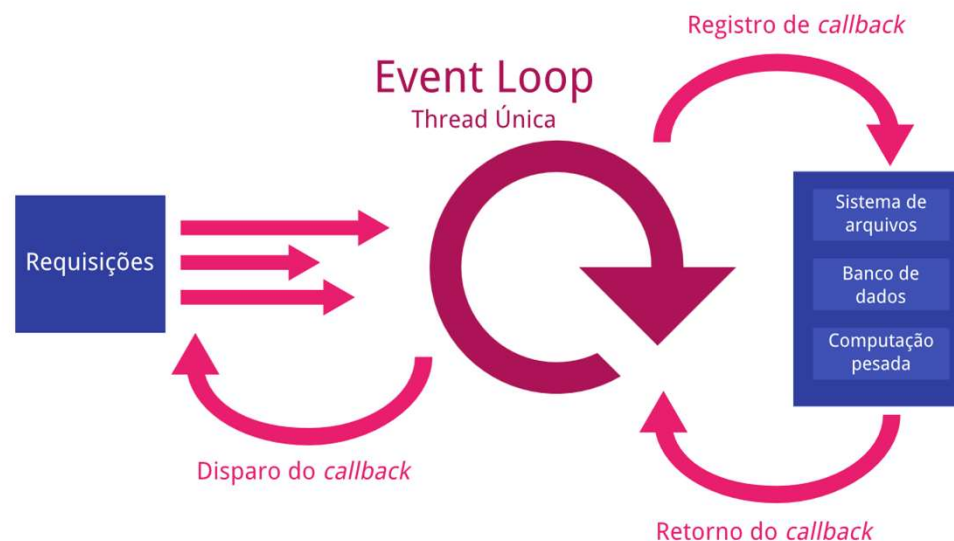
- Um motor de execução otimizado e rápido, construído sobre o [V8 da Google](https://v8.dev/)
- Código executado de forma assíncrona por padrão — nada é bloqueante;
- Projeto baseado em iteração de eventos de forma parecida com os navegadores modernos;
- Acesso a rede como prioridade, você pode criar um servidor web com algumas poucas linhas de código;
- Uma potente biblioteca de streams;
- Um sistema gerenciador de pacotes amigável com alguns milhares de módulos open-source para escolha, o [NPM](https://www.npmjs.com/).



NODE JS

Como o Node funciona

O Node trata conexões de forma diferente, uma única thread recebe todas as conexões, ou seja, não há concorrência de recursos. Essa única *thread*, chamada de **Event Loop** (que podemos traduzir para laço de eventos), controla todos os outros fluxos assíncronos. Assim, o Node elimina o gargalo de um máximo de requisições que os servidores convencionais sofrem





Como o Node funciona

Quais as vantagens do Node.js?

- Node.js usa Javascript
- Node.js permite Javascript full-stack
- Node.js é muito leve (pouco investimento em infraestrutura) e é multiplataforma

Quais as desvantagens do Node.js?

- Node.js usa Javascript (tipagem fraca, POO fora do padrão)
- Node.js é recente (2009)
- Node.js é assíncrono (uso demasiado de call-backs, não recomendado para aplicações que exige muitos cálculos)



NODE JS

Baixar o Node.js®

Baixe o Node.js® v24.18.0 LTS para Windows usando Docker com npm

Informação Quer novos recursos mais cedo? Obtenha a [versão mais recente do Node.js](#) e experimente as últimas melhorias!

```
1 # O Docker possui instruções específicas de instalação para cada sistema operacional.
2 # Consulte a documentação oficial em https://docker.com/get-started/
3
4 # Baixar a imagem Docker do Node.js:
5 docker pull node:24-alpine
6
7 # Criar um contêiner do Node.js e iniciar uma sessão Shell:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verifique a versão do Node.js:
11 node -v # Deve exibir "v24.18.0".
12
13 # Verificar a versão do npm:
14 npm -v # Deve imprimir "11.16.0".
```

PowerShell

</> Copiar para a área de transferência

Docker é uma plataforma de containerização. Se você encontrar algum problema, por favor visite o [site do Docker](#)

Ou baixe uma versão pré-compilada do Node.js® para Windows rodando uma arquitetura x64.

Instalador Windows (.msi)

Binário Independente (.zip)

**Instale essa
versão**

PARA INSTALAR O NODE EM SEU COMPUTADOR ENTRE NO SEGUINTE LINK <https://nodejs.org/pt-br/download> , BAIXE O ARQUIVO E FAÇA A INSTALAÇÃO NO SEU COMPUTADOR.

Professor Flávio Mota



NODE JS

Para rodar qualquer arquivo .js no **node**, abra o terminal de comandos do Windows ou o terminal do Visual Studio Code, posicione a linha de comando onde está localizado seu arquivo .js e digite o seguinte comando : **node arquivo.js** e <enter>.

The screenshot shows the Visual Studio Code interface. The top editor pane displays a file named `teste.js` with the following content:

```
C: > Users > Flavio Mota > Desktop > JS teste.js
1 console.log("Testando NodeJS - Professor Flávio Mota")
```

The bottom panel shows the **TERMINAL** tab with the following commands and output:

```
PS C:\Users\Flavio Mota> node -v
v12.16.0
PS C:\Users\Flavio Mota> cd desktop
PS C:\Users\Flavio Mota\desktop> node teste
Testando NodeJS - Professor Flávio Mota
PS C:\Users\Flavio Mota\desktop>
```

Red boxes highlight the file path and content in the editor, and the command execution in the terminal. Red arrows point from the explanatory text to these elements.

Arquivo teste.js aberto no visual studio code

Resultado no terminal do visual studio code, após executar o arquivo teste.js



NODE JS

Executando Javascript no Node

```
function soma(x,y) {  
    return x + y  
}  
function subtracao(x,y) {  
    return x - y  
}  
function multiplicacao(x,y) {  
    return x * y  
}  
function divisao(x,y) {  
    return x / y  
}  
console.log(soma(10,5))  
console.log(subtracao(10,5))  
console.log(multiplicacao(10,5))  
console.log(divisao(10,5))
```

Calculos.js

```
PS E:\Desktop Antigo\PROJETO RECODE\AULAS\Programe Back end\NodeJS\Calculos> node Calculos  
15  
5  
50  
2  
PS E:\Desktop Antigo\PROJETO RECODE\AULAS\Programe Back end\NodeJS\Calculos>
```

Saída no terminal

Executando um arquivo
Javascript no NODE



NODE JS

Módulos: exports e require()

Módulo soma.js

```
let soma = function(x, y) {  
  return x + y  
}  
  
module.exports = soma
```

Módulo multiplicação.js

```
let mult = function(x, y) {  
  return x * y  
}  
  
module.exports = mult
```

Módulo subtração.js

```
let subtr = function(x, y) {  
  return x - y  
}  
  
module.exports = subtr
```

Módulo divisão.js

```
let divisao = function(x, y) {  
  return x / y  
}  
  
module.exports = divisao
```



NODE JS

Módulos: exports e require()

Módulos são cruciais para construção de aplicações em Node pois eles permitem incluir bibliotecas externas, como bibliotecas de acesso ao banco de dados, e ajudam a organizar seu código em partes separadas com responsabilidades limitada. Utilizar módulos em Node é simples, você usa a função **require()**, que recebe um argumento: o nome da biblioteca do core ou o caminho do arquivo do módulo que você quer carregar.

Estrutura do projeto anterior dividido em módulos

```
JS calculadora.js
JS divisao.js
JS multiplicacao.js
JS soma.js
JS subtracao.js
```

Arquivo calculadora.js importando os módulos

```
const subtr = require('./subtracao')
const soma = require('./soma')
const mult = require('./multiplicacao')
const divisao = require('./divisao')
console.log(soma(40,2))
console.log(subtr(40,2))
console.log(mult(40,2))
console.log(divisao(40,2))
```

```
node calculadora
```

Execute no terminal o módulo principal calculadora



NODE JS

Criando nosso primeiro servidor em NODE

Arquivo servidor.js

```
var servidor = require('http');  
  
servidor.createServer(function(req, res){  
  res.end("<h1> Ola Mundo!!</h1>")  
}).listen(8001);  
  
console.log('Servidor subiu')
```

Módulo http sendo importado e atribuído a variável servidor

Método `createServer()` cria um novo servidor com uma função call-back que recebe dois parâmetros 'req' e 'res', representando as requisições e respostas do servidor. O método `listen(8001)` representa a porta que o servidor está sendo escutado.

O método `end` do objeto `res` e a devolução visual no navegador

```
PS E:\Desktop Antigo\PROJETO RECODE\AULAS\Programe Back end\NodeJS\AppServidor> node servidor  
Servidor subiu
```

Execute o arquivo no terminal.

Ola Mundo!!

<http://localhost:8001/> servidor rodando neste endereço local



NODE JS

Criando rotas em node

Arquivo servidor2.js

```
var http = require('http');

var server = http.createServer(function(req, res){
  var categoria = req.url;

  if(categoria == '/front-end'){
    res.end("<html><body>Tecnologias Front-End: TypeScript, Angular, React..</body></html>");
  }else if(categoria == '/back-end'){
    res.end("<html><body>Tecnologias Back-End: NodeJS, Python, PHP, MySQL...</body></html>");
  }else if(categoria == '/infraestrutura'){
    res.end("<html><body>Azure Cloud, Linux, MySQL Server...</body></html>");
  }else{
    res.end("<html><body>Programador Full Stack</body></html>");
  }
});

server.listen(3000);
```



NODE JS

Criando rotas em node

```
var http = require('http');
var fs = require('fs');
var server = http.createServer(function (request, response) {
  var categoria = request.url;
  // A constante __dirname retorna o diretório raiz da aplicação.
  if (categoria == '/index') {
    fs.readFile(__dirname + '/index.html', function (err, html) {
      response.end(html);
    });
  } else if (categoria == '/teste') {
    fs.readFile(__dirname + '/teste.html', function (err, html) {
      response.end(html);
    });
  }
});
server.listen(3000, function () {
  console.log('Executando Site Pessoal');
});
```

Rota para index.html

localhost:3000/index

Bem vindo ao meu site pessoal

Rota para teste.html

localhost:3000/teste

Bem vindo a pagina de teste

Arquivo servidor3.js

Professor Flávio Mota



Trabalhando com o framework EXPRESS

O Express é um framework para aplicativo da web do Node.js mínimo e flexível que fornece um conjunto robusto de recursos para aplicativos web e móvel.

Vamos criar um servidor usando o express

*1- passo : instalar o framework em seu projeto **npm install express --save** , com esse comando o framework ficará instalado em seu projeto, caso queira instalar temporariamente para testes esse pacote faça o mesmo comando sem a clausula **--save***

NODE JS - EXPRESS



Trabalhando com o framework EXPRESS

2 - passo : crie um objeto chamado **app** que representa uma instancia do expresss no seu projeto, crie uma rota inicial usando o método GET e configure a porta que o servidor será escutado conforme o exemplo abaixo

```
const express = require('express')
const app = express()

app.get("/", (req, res) => {
  res.send("Professor Flávio Mota")
})

app.listen(3000)
console.log('Servidor subiu')
```

Objeto app representando
o express

Configuração da Rota raiz

Porta que o sistema vai
responder

O aplicativo inicia um servidor e escuta a porta 3000 por conexões. O aplicativo responde com "Professor Flávio Mota" à solicitações para a URL raiz (/) ou **rota**. Para todos os outros caminhos, ele irá responder com um **404 Não Encontrado**.



Explicando o Roteamento do nosso app

O **Roteamento** refere-se à determinação de como um aplicativo responde a uma solicitação do cliente por um endpoint específico, que é uma URI (ou caminho) e um método de solicitação HTTP específico (GET, POST, e assim por diante). Cada rota pode ter uma ou mais funções de manipulação, que são executadas quando a rota é correspondida.

A definição de rotas aceita a seguinte estrutura:

app.METHOD(PATH, HANDLER)

Onde:

- **app** é uma instância do express.
- METHOD **get** é um método de solicitação HTTP.
- PATH **"/"** é um caminho no servidor.
- HANDLER (**req, res**) é a função executada quando a rota é correspondida.

```
app.get("/", (req, res) => {  
  res.send("Professor Flávio Mota")  
})
```




Métodos HTTP

Podemos criar outros tipos de rotas usando o **express** e os métodos do protocolo HTTP, para criamos nossas APIs além do método GET podemos usar o POST, PUT e DELETE

- GET - recupera os dados identificados pela URL, requisições GET fazem com que o servidor devolva algo para o cliente, algo que estar “dentro” do servidor.
- POST - cria um novo recurso, significa que estamos querendo incluir algum recurso no servidor.
- PUT - atualiza um recurso, significa que estamos querendo atualizar algum recurso no servidor.
- DELETE - apaga um recurso, significa que estamos querendo apagar algum recurso no servidor.



Métodos HTTP

- **GET** - recupera os dados identificados pela URL, requisições GET fazem com que o servidor devolva algo para o cliente, algo que estar “dentro” do servidor.

```
app.get("/", (req, res) => {  
  res.send("Professor Flávio Mota")  
})
```



Métodos HTTP

- **POST** - cria um novo recurso, significa que estamos querendo incluir algum recurso no servidor.

```
app.post('/frontend/Salva', function(req, res) {  
    var dados = req.body;  
  
    connection.query('INSERT INTO conteudo SET ?', dados, function(error, result) {  
        res.redirect('/frontend');  
    });  
});
```



Métodos HTTP

- PUT - atualiza um recurso, significa que estamos querendo atualizar algum recurso no servidor.

```
app.put('/conteudo/:id', (req, res) => {  
  const { index } = req.params; // recupera o index com os dados  
  const { name } = req.body;  
  
  dados[index] = name; // sobrepõe o index obtido na rota de acordo com o novo valor  
  
  return res.json(dados);  
}); // retorna novamente os dados atualizados após o update
```



Métodos HTTP

- DELETE - apaga um recurso, significa que estamos querendo apagar algum recurso no servidor.

```
app.delete('/conteudo/:id', (req, res) => {  
  const { index } = req.params; // recupera o index com os dados  
  
  geeks.splice(index, 1); // percorre o vetor até o index selecionado e deleta uma posição no array  
  
  return res.send();  
}); // retorna os dados após exclusão
```

NODE JS - EXPRESS



Roteamento básico – renderizando arquivos HTML

```
const express = require('express')
const app = express()

app.get("/", (req, res) => {
  res.sendFile(__dirname + "/src/artigos.html")
})

app.get("/artigos", (req, res) => {
  res.sendFile(__dirname + "/src/artigos.html")
})
```

```
app.get("/contato", (req, res) => {
  res.sendFile(__dirname + "/src/contato.html")
})

app.get("*", (req, res) => {
  res.status(404).sendFile(__dirname + "/src/erro.html")
})

app.listen(3333)
console.log('up')
```

Nesse exemplo estamos renderizando arquivos HTML de acordo com a rota solicitada no navegador, a palavra reservada **__dirname** indica a pasta raiz da aplicação o endereço **/src/artigos.html** é o local do arquivo HTML